

WisePIFinder: Efficient and Accurate Detection of Persistent and Infrequent Flows

Zengxie Ma*, Yao Xin[†], Ying Chen*, Zhuochen Fan[‡], Tong Li*, Ning Hu[‡], Qing Liao[§] [‡], Yi Zhao[¶], Feng Zhang*

*Renmin University of China [†]Guangzhou University [‡]Pengcheng Laboratory

[§]Harbin Institute of Technology(Shenzhen) [¶]Beijing Institute of Technology

Email: {zengxie.ma, chenying1024, tong.li, fengzhang}@ruc.edu.cn;

{fanzhch, hun}@pcl.ac.cn; liaoqing@hit.edu.cn; zhaoyi@bit.edu.cn

Abstract—In large-scale data stream analytics, accurate identification of Persistent and Infrequent (PI) flows is of great significance for monitoring and protecting against network attacks such as Advanced Persistent Threats (APT). However, existing research focuses mainly on detecting frequent flows or persistent flows, with insufficient studies on the characterization and detection methods for PI flows. Based on the analysis of sufficient APT flows, we propose a method that combines global and local features to effectively characterize PI flows. Further, we propose a novel sketch algorithm called WisePIFinder, which aims to detect PI flows more accurately and efficiently in real-time. The key idea is to continuously filter out non-PI flows while detecting flow persistence, to achieve accurate statistics on PI flows. Experimental results show that WisePIFinder improves the F1 Score by at least 20% and insertion throughput by at least 60% compared to the state-of-the-art solution for detecting PI flows. All related codes have been open-sourced on GitHub.

Index Terms—Sketch, Data Streams, PI Flows, Detection, Advanced Persistent Threats

I. INTRODUCTION

A. Background

With the explosive growth in network users and scale, network data streams exhibit unprecedented complexity and unpredictability. In-depth analysis of the characteristics in these data streams, especially the detection of key ones such as elephant flows and persistent flows, has become fundamental for improving network performance and optimizing network infrastructure [1]. Among massive data streams, a special type—Persistent and Infrequent Flow (PI flow)—has gradually attracted the attention of the research community. Although these flows exhibit relatively low traffic volume per unit time, they demonstrate significant characteristics of long duration and high stability [2], [3].

PI flows have broad applications across critical domains, including network security and system monitoring. In network security, Advanced Persistent Threats (APT) [4] and DNS tunnel attacks [5] often employ persistent covert methods

to infiltrate target systems. These attacks evade traditional traffic monitoring systems through long-term, low-profile data transmission, with traffic behavioral patterns highly consistent with PI characteristics. Accurate identification of such traffic patterns is crucial for preventing network attacks and reducing system security risks. In large-scale distributed systems, node state synchronization mechanisms implemented through periodic heartbeat packets also naturally exhibit PI characteristics [6], [7]. Accurately monitoring such traffic not only helps evaluate the system's operational status but also enables the timely detection of potential system anomalies. In seismological research, infrequent seismic wave signals serve as unique early warning indicators for seismic activity. These signals not only propagate over long distances but can also penetrate complex geological structures [8], with propagation characteristics significantly similar to PI patterns. Real-time monitoring and analysis of these signals can provide a critical decision-making basis for earthquake warning systems. Furthermore, in social network analysis, user interaction patterns in PI often reflect stable social network relationships. These long-term but low-frequency interaction behavioral patterns provide important research perspectives to understand the evolution of social networks and user behavior characteristics.

Sketch, as a type of probabilistic data structure, is undoubtedly one of the best solutions for data stream analysis-related tasks. Compared with traditional data storage methods, Sketch not only demonstrates significant advantages in storage efficiency but also maintains high query accuracy while keeping the time complexity of data writing and querying operations at constant levels. Classical Sketch structures such as CM Sketch [9], achieve efficient data compression storage through ingenious design. However, these traditional designs are mainly optimized for per-flow size estimation or large-flow detection scenarios and show notable limitations in detecting PI flows.

The characteristics of PI flow networks exhibit sparsity and infrequency. In scenarios with limited memory resources, effectively detecting PI flows from high-speed, massive data streams has become a critical issue that urgently needs to be addressed. The existing sketch algorithms for flow persistence estimation, such as PIE [10] and On-Off Sketch [11], can

This work is supported in part by in part by the National Natural Science Foundation of China (NSFC) (No.62202473, 62272466, 62441230, 62372123, 62402012, 62472036 and 62202258), and the Major Key Project of PCL (Grant No. PCL2024A05). Corresponding author: Tong Li and Yao Xin.

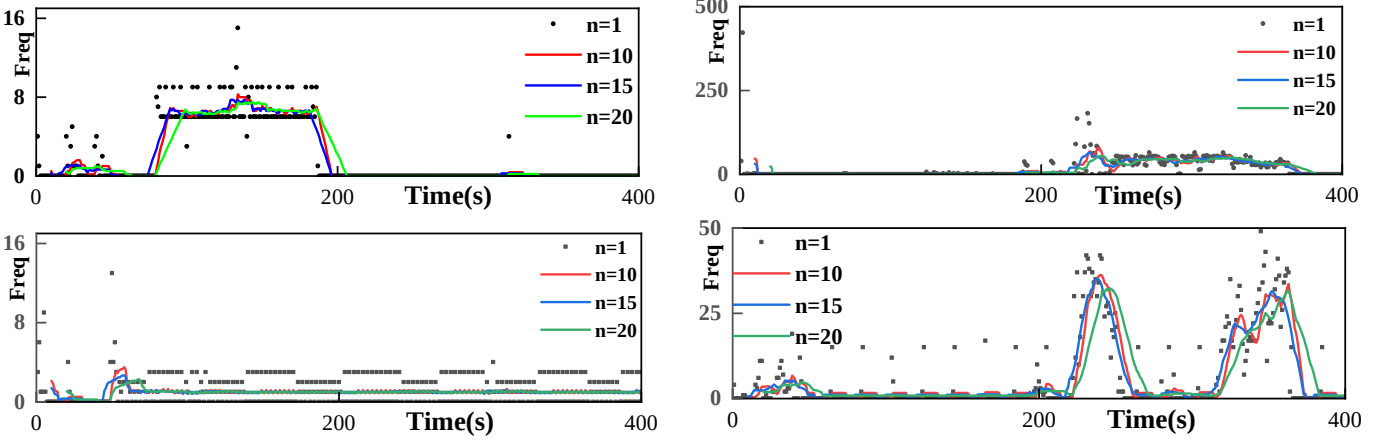


Fig. 1: Frequency of the APT flow in each observation window and the SMA of the continuous n observation windows.

effectively estimate flow persistence, but they are not efficient when used forcibly for PI flow identification. Current research on Sketch primarily focuses on either frequency estimation or persistence analysis¹, and few studies can effectively unify these two dimensions.

The first Sketch for PI flow detection, PISketch [2], [3], attempts to address this problem by introducing a weighting mechanism, but its definition of PI flow using weights is still somewhat limited. Additionally, PISketch uses Bloom filter combined with sketch structure to update weights and maintain high-weight PI flows, but it requires two steps to complete the detection task, and the overall performance needs improvement. Solutions that directly combine persistence detection with frequency estimation algorithms (such as On-Off + CM Sketch and PIE + CM Sketch) are faced with the issues of high memory overhead and low processing speed.

B. Our Proposed Solution

Based on the analysis of real-world network traffic, including heartbeat packets and APT attack traffic, we propose a novel method to more accurately define PI flows in data streams, which not only considers global traffic characteristics but also introduces local feature analysis for the first time. In other words, PI flows are characterized from both global and local perspectives. At the global level, the Average Packet Rate (APR) is used to represent the average packet rate within each occurrence window. In contrast to traditional frequency estimation methods, APR is unaffected by the total duration of the measurement task and can be stored using fixed byte sizes (e.g., half-precision floating-point types). At the local level, the Simple Moving Average (SMA) is applied to describe short-term behavioral patterns across consecutive windows. This dual-dimensional analysis effectively addresses PI flow misidentification in existing solutions.

For the complex network measurement task of PI flow detection that we are concerned about, WisePIFinder only

requires a customized single-layer hash bucket array to accomplish this task, unlike the “multi-layer data structure + multi-stage processing” approach that state-of-the-art PISketch relies on. Specifically, WisePIFinder efficiently calculates the APR and SMA of any given data stream and continuously eliminates non-PI flows based on these two metrics. Then, by continuously sorting flows according to their persistence, WisePIFinder gradually pushes candidate PI flows to the top of each hash bucket. Thanks to this design, we can achieve more accurate PI flow detection without using the Bloom filter. Compared with existing solutions including PISketch, WisePIFinder significantly improves execution efficiency and memory utilization and reduces computational complexity. We provide the related code open-source at GitHub [12]. Our main contributions include:

- We propose a more comprehensive description of PI flow characteristics, quantifying traffic characteristics through APR and SMA from both global and local dimensions, providing new directions for related research.
- At the algorithmic level, we design the WisePIFinder framework based on a single-layer hash bucket array. By optimizing data structures and processing flows, we enhance the algorithm’s execution efficiency while ensuring detection accuracy. Theoretical analysis also confirms the framework’s advantages in accuracy.
- We conduct performance evaluations of the algorithm on three representative real-world datasets. The experimental results show that, compared to existing approaches, WisePIFinder improves detection accuracy and insertion throughput by at least 20% and 60%, respectively, validating the practical value of this method.

The rest of this paper is organized as follows: § II discusses the motivation for the proposed work and defines the PI flow in detail. § III details the design principles and implementation of WisePIFinder, including its key operations and optimization strategies. § IV analyzes algorithm performance from a theoretical perspective. § V demonstrates system performance through experimental results. § VI introduces related work, providing an overview of previous studies and existing meth-

¹Frequency (flow size) refers to the number of occurrences of a flow, and persistence refers to the number of windows in which a flow appears.

ods in the field. Finally, § VII concludes the paper.

II. MOTIVATION AND PROBLEM DEFINITION

A. Analysis of PI Flow

PI flows manifest in various typical forms in real network environments, with APT attack traffic and system heartbeat communications being most representative [3]. Heartbeat packets, used as a system-to-system alive detection mechanism, typically send fixed-size packets at fixed time intervals, making them ideal subjects for studying PI flow characteristics. In contrast, APT attacks exhibit more complex traffic patterns. Attackers usually maintain control over target systems through long-term but intermittent low-speed data transmission, characteristics highly consistent with the definition of steady flows [13]. Through systematic literature review and traffic characteristic analysis, we find that APT flows rarely exhibit high-frequency behavior in consecutive time windows, typically appearing once every several windows, thus maintaining consistently low SMA (refer to Equation (3)) levels.

Fig.1 illustrates representative traffic distribution patterns from the Mediafire APT dataset [14], where 1-second time windows are plotted along the x-axis and flow frequency along the y-axis. Black dots represent actual frequency values per window, while the overlaid curves show SMA computed using different window sizes (10, 15, and 20 windows in red, blue, and green, respectively). These SMA curves exhibit consistent behavioral patterns across different window sizes.

B. Motivation

Current research works fail to provide comprehensive descriptions or definitions of PI flows that adequately capture the characteristics of real network traffic, leading to significant detection misses. For instance, the weight-based description proposed in PISketch lacks an intuitive reflection of PI flow characteristics. Specifically, the relationship between algorithmic weights and actual traffic physical features can be described as follows:

$$f \leq (L + 1) \cdot N - \tau \quad (1)$$

where L and τ are user-defined parameters, and N represents the cumulative appearance count of flows in different windows. As shown in Fig.2(a), the x-axis represents the cumulative window count of flow appearances (persistence measure), and the y-axis represents the total frequency.

The blue triangular region in Fig.2(a) represents the PI flow identification area, where point B 's N_p value corresponds to the persistence threshold—flows are only considered for detection when N exceeds N_p . Experimental analysis shows that low-activity heartbeat packets from Fig.2(a) are typically distributed in the region S_1 of Fig.2(a), and are missed despite exhibiting PI flow characteristics due to insufficient window size or insufficient number of window appearances. Additionally, burst flows and flows with significant frequency variations in the PI flow detection region (S_2 area of Fig.2(a)), are misclassified due to their high weight values.

The current detection algorithms are unable to simultaneously and effectively tackle both of the issues mentioned above. Adjusting the slope or threshold τ to expand detection coverage to the S_1 region would increase misclassification rates for other traffic types, reducing detection accuracy. Increasing the weight threshold τ to reduce burst flow misclassification would result in more PI flow misses, lowering the recall rate. Based on these observations, we propose WisePIFinder to enhance both PI flow detection accuracy and processing efficiency.

C. Definition & Problem Statement

To comprehensively capture the characteristics present in real network traffic, we quantify "Infrequency" from two dimensions and propose three quantitative metrics for a detailed PI flow definition.

- **Persistence:** Network traffic follows Zipf distribution [15], with 20% being elephant flows and 80% being mouse flows. Most small flows are short-term one-time traffic, while PI flows appear persistently across multiple windows. For flow e in the data stream, its persistence is defined as the number of appearances of e in T non-overlapping consecutive time windows [11].
- **Average Packet Rate (APR):** APR represents the average packet rate within each occurrence window. Research shows that PI flows' frequency distribution typically fluctuates within a small range around their average rate, making this metric effective in reflecting PI flows' mean characteristics.

$$\nu = \frac{f}{N} \quad (2)$$

where f represents total flow frequency and N represents total number of windows where flow appears. In fact, we use a more memory-efficient method for the calculation:

$$\nu_i = \begin{cases} 0 & i = 0 \\ \frac{\nu_{i-1} * (N-1) + C_i}{N} & i > 0 \end{cases} \quad (3)$$

where C_i is the frequency of occurrence in the i -th window.

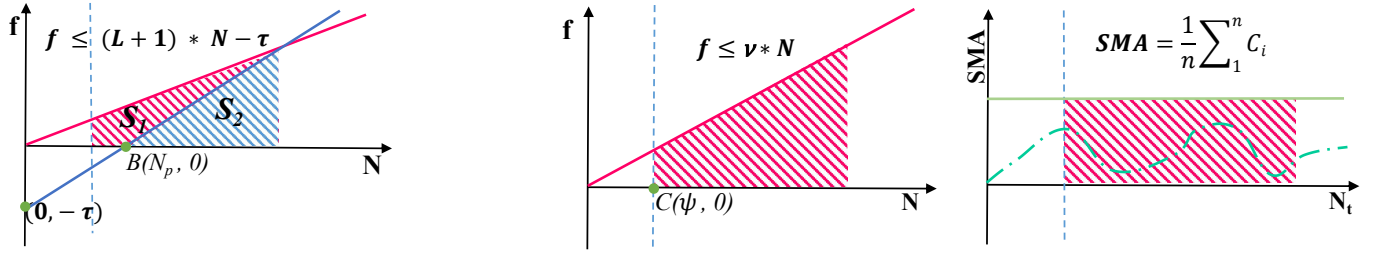
- **Simple Moving Average (SMA):** SMA is the arithmetic average of the multi-window frequency, which can reduce random fluctuation effects and smooth data through sliding calculation windows to highlight long-term traffic trends.

$$\text{SMA} = \frac{1}{n} \sum_{i=1}^n C_i \quad (4)$$

where n is the number of consecutive observation windows, which can be set as required.

The APR is independent of measurement duration, allowing for efficient storage in 2-byte fixed memory. Meanwhile, SMA offers both computational simplicity and effective filtering of noise and transient fluctuations, enabling it to accurately characterize long-term flow behavior patterns.

Fig.2(b) illustrates the core concept of WisePIFinder: the left graph illustrates the distribution of APR for the PI flow over historical windows, while the right graph depicts the variation trends of the rate over time. Given persistence



(a) PISketch's detectable range S_2 and undetectable range S_1

(b) More precise definition of PI: $\{(\nu \leq \omega) \cap (N \geq \psi)\} \cap (SMA \leq \phi)$

Fig. 2: Comparison of the PI flow definition scope between PISketch and WisePIFinder.

requirements are met, the red region in the left graph (including S_1 and S_2) identifies candidate flows with low APR, effectively solving PISketch's missing detection issue. The red region in the right graph represents candidate flows with low SMA, with the x-axis indicating consecutive time windows, effectively solving the misdetection problem of PISketch. By simultaneously identifying flows with both low APR and low SMA characteristics, our algorithm effectively resolves both the missing detection and misclassification issues in PISketch. Table I summarizes the main symbols and their descriptions.

TABLE I: Frequently-used notations in this paper

Notation	Description
V	Total number of time windows
Id	Unique identifier for data streams
N	Total number of windows where flow appears
d	Depth of hash buckets
ν	Average Packet Rate
S	New window indicator (False: new window)
C	Frequency of flow in current window
n	Number of consecutive observation windows
M	Total frequency in n consecutive windows
ω	Maximum threshold for APR
ψ	Minimum threshold for window appearances
ϕ	Maximum threshold for SMA

III. DESIGN OF WISEPIFINDER

A. The WisePIFinder

Rationale: In this paper, we propose a novel algorithm, namely WisePIFinder. The measurement task duration is divided into evenly spaced time windows, and WisePiFinder quickly calculates the APR and SMA of flows in each, rapidly excluding frequent flows. It then uses an ordered data structure to push more persistent flows to the top of each hash bucket, with PI flow detection applied to the infrequent ones.

Data Structure: As shown in Fig.3, the WisePIFinder data structure consists of two collaborative counters: GlobalCounter, which records overall characteristics of the flow (e.g., identifiers and persistence), and LocalCounter, which tracks local features (e.g., frequency of the current window). The two counters collaborate through one-to-one mapping relationships. Since the information in the local features needs to be periodically updated and to optimize the algorithm's execution efficiency, we separate the LocalCounter and use an overall structure with continuous data memory.

GlobalCounter contains two hash bucket arrays (A_1 and A_2), which help reduce the collision rate by utilizing two

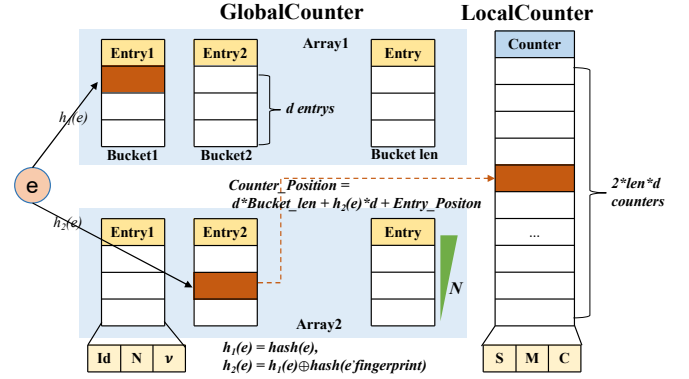


Fig. 3: The data structure of WisePIFinder

separate hash buckets. Each bucket entry stores three elements: Id (flow identifier, computed from the five-tuple of source/destination addresses, ports, and protocol), N (total number of appeared windows, i.e., persistent), and ν (APR, calculated as total frequency divided by N). Entries within each bucket are always sorted in descending order, with the largest N placed at the top.

LocalCounter implements a two-dimensional array with a counter (each row) containing three elements: S (new window indicator, reset to False at window boundaries and set to True upon first flow appearance), M (cumulative frequency over past n windows), and C (frequency of the current window). SMA does not need to be directly calculated, as it can be equivalently represented by M under the condition that n is fixed, thereby reducing the need for division operations. Furthermore, since PI flows typically do not appear in most windows, we perform sampling at equal intervals on the SMA to exclude non-PI flows, thereby further enhancing the algorithm's efficiency.

Unlike traditional multi-layer sketch designs, WisePiFinder uses a single-layer structure for compound task detection. This approach avoids the computational complexity and memory overhead of multi-layer solutions like PISketch (e.g., Bloom Filter with Sketch).

B. Operations

Insertion: During initialization, both GlobalCounter and LocalCounter are set to zero. The system periodically clears S and M in the LocalCounter based on the requirements of the measurement tasks, with S cleared at the start of each new window and M cleared all n windows. When flow e appears

in the i -th time window, the system first calculates two hash values $h_1(e)$ and $h_2(e)$ to locate candidate buckets $A_1[h_1(e)]$ and $A_2[h_2(e)]$. The flow e will exist only in $A_1[h_1(e)]$ or $A_2[h_2(e)]$, but not both. Traversing entries in both buckets may result in three scenarios. The pseudocode for this process is presented in Algorithm 1.

- When a matching entry is found, indicating the flow exists, and then we check the information of this flow.
 - 1) When S is **False**, indicating the first appearance in the current window:
 - $\nu \leq \omega$. In the GlobalCounter, update N and ν . Meanwhile, in the LocalCounter, set S to **True**, and update C and M .
 - $\nu > \omega$. No operation.

After updating N , compare it with the previous entry's N . Use bubble sort to ensure that flows with higher persistence are placed at the top of the heap while maintaining the large ν values below when they are equal.

- 2) When S is **True**, indicating that the flow does not appear for the first time, only accumulates C and M .
- 3) If $M > \phi * n$, indicating that the flow is too active and does not match the characteristics of the PI flow, update $\nu = \phi * n$.
- If no matching entry exists, but an empty entry is available, insert the flow information directly.
- If no matching entry and no empty entry available:
 - 1) If an entry in the bucket has $\nu > \omega$, it can be replaced.
 - 2) Otherwise, execute decay strategy:
 - No operation is performed when $N = 1$, $\nu = 0$, and $S = \text{False}$ to protect potential PI flows.
 - Other cases: $\nu = \nu - \alpha$, remove flows with smaller N when $\nu < 0$. The decay factor α is user-defined.

Query: WisePIFinder preserves potential PI flow information through the mechanisms outlined above. During queries, the system traverses hash buckets and extracts entries that meet the threshold conditions. Note that the flows in the query results are PI flow candidates, which require further verification by the user. Additionally, since the number of reported PI flows is directly related to the memory size of the data structure, appropriate memory configuration should be determined based on the expected minimum number of PI flows in the actual deployment.

Example: Fig.4 demonstrates WisePIFinder's state transition process during different data stream insertions. Experimental parameters are set to $d=4$, $n=10$, $\omega=20$. The system first determines GlobalCounter and LocalCounter positions through $h_1(e)$ or $h_2(e)$. For flow e_1 , since the LocalCounter's flag indicates a non-first appearance, only the accumulation operations for M and C are executed. When flow e_2 first appears in the current window, N and ν will be updated, while also updating M and C in the LocalCounter. When flow e_4 first appears, besides updating basic information, its N value (11) exceeds that of previous entry (flow e_3), triggering position exchange to maintain flows with larger N values at

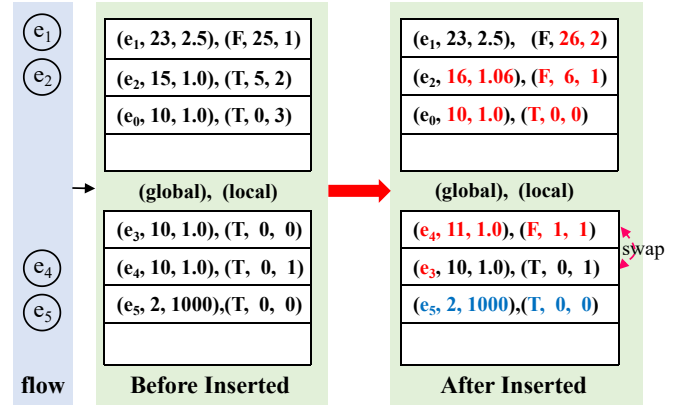


Fig. 4: Examples of flows inserted into the WisePIFinder heap top. The ν value (1000) of flow e_5 exceeds threshold ω , so no update operation is performed. This mechanism ensures PI flows tend to distribute at stack top while effectively excluding large flows and one-time flows.

Algorithm 1 Insert(e): Insert a flow e into WisePIFinder

Input: Flow e to be inserted

Notation:

- $A_i[h][k]$: Entry k in bucket h of array i , where $i \in (1, 2)$
- A_e : Entry corresponding to flow e in array A_i
- UC_e : Counter for flow e in LocalCounter
- Clear S per window, M per L windows

```

1:  $fp \leftarrow \text{fingerprint}(e)$ 
2:  $h_1 \leftarrow \text{hash}(e)$ ,  $h_2 \leftarrow h_1 \oplus \text{hash}(fp)$ 
3: if  $e \in \{A_1[h_1] \cup A_2[h_2]\}$  then
4:   if  $A_e.\nu > \omega$  then
5:     return
6:   end if
7:   if isFirstOccurrence( $e$ ) then
8:      $A_e.\nu \leftarrow (A_e.\nu * A_e.N + UC_e.C) / (A_e.N + 1)$ 
9:      $A_e.N \leftarrow A_e.N + 1$ 
10:    sortPersistent( $A_e$ )
11:   end if
12:    $UC_e.C \leftarrow UC_e.C + 1$ 
13:    $UC_e.M \leftarrow UC_e.M + 1$ 
14:   if  $UC_e.M > \phi * n$  then
15:      $A_e.\nu \leftarrow \phi * n$ 
16:   end if
17: else if hasEmptyEntry( $A_1[h_1] \cup A_2[h_2]$ ) then
18:   insertNewEntry( $A_e, UC_e$ )
19: else if hasOverflowEntry( $A_1[h_1] \cup A_2[h_2], \omega$ ) then
20:   insertNewEntry( $A_e, UC_e$ )
21: else
22:   decayLastEntry()
23: end if

```

IV. MATHEMATICAL ANALYSIS

Based on the implementation process of WisePIFinder, we first analyze the properties of the ν and SMA, followed by an evaluation of the algorithm's accuracy and recall, thus

demonstrating the algorithm's effectiveness. Table II lists the main symbols used in this section and their descriptions.

TABLE II: Symbols Used In Mathematical Analysis

Notation	Description
T	Number of Flows
D	Number of buckets
ρ	The probability of appearing in a window

Let X_i denote whether it appears in the i -th window and X_i follows a Bernoulli distribution, i.e.,

$$P(X_i) = \begin{cases} \rho & X_i = 1 \\ 1 - \rho & X_i = 0 \end{cases} \quad (5)$$

When $X_i = 1$, C follows a Poisson distribution with parameter λ , as follows:

$$N = \sum_{i=1}^V X_i, f = \sum_{i=1}^V C_i X_i \quad (6)$$

X_i and C_i are independent and uncorrelated. Since $N \sim B(n, p)$, we can deduce that:

$$EN = V * \rho \quad (7)$$

A. Property of ν

Theorem 1: The ν is an unbiased estimate of the flow frequency, independent of the probability of its appearance in the window, i.e., $E\nu = \lambda$

Proof: Both f and N are random variables, and conditional expectation is used to solve the problem.

$$E\nu = E\left(\frac{f}{N}\right) = E\left(E\left(\frac{f}{N} \middle| N\right)\right) \quad (8)$$

Given N ,

$$\begin{aligned} E\left(\frac{f}{N} \middle| N = n\right) &= E\left(\frac{\sum_{i=1}^V C_i * X_i}{N} \middle| N = n\right) \\ &= \frac{1}{n} E \sum_{i=1}^V C_i X_i = \frac{1}{n} \sum_{i=1}^V \lambda * EX_i = \lambda \end{aligned} \quad (9)$$

For any N , $E\left(\frac{f}{N} \middle| N\right) = \lambda$, So,

$$E\left(E\left(\frac{f}{N} \middle| N\right)\right) = E\lambda = \lambda \quad (10)$$

B. Property of SMA

Theorem 2: The $E(SMA)$ is the product of the probability of occurrence in the window and the average number of occurrences per window, i.e.,

$$E(SMA) = \rho * \lambda \quad (11)$$

Proof:

$$SMA = \frac{\sum_{i=1}^n C_i}{n} = \frac{\sum_{i=1}^n C_i * X_i}{n} \quad (12)$$

$$E(SMA) = E\left(\frac{\sum_{i=1}^n C_i * X_i}{n}\right) = \frac{1}{n} \sum_{i=1}^n \lambda * EX_i = \rho * \lambda \quad (13)$$

The smaller the probability ρ of appearing in the window, the smaller the $E(SMA)$ will be. If $\rho = 1$, appearing in all windows, then $E(SMA) = \lambda$. As

$$n \rightarrow \infty, SMA \approx \nu \quad (14)$$

C. Error bound of Hash

Assume that each bucket has d entries. If a bucket is assigned more than d PI flows, some PI flows will be missed, affecting the recall rate. The following theorem ensures that the impact is constrained within a bound in the hash phase.

Theorem 3: If M flows are uniformly distributed to D buckets by the hash function, the probability that at least one bucket is assigned more than d PI flows is:

$$P < D \left(\frac{e^\delta}{(1 + \delta)^{(1 + \delta)}} \right)^{\frac{M}{D}}, \delta = \frac{d}{\lambda} - 1 \quad (15)$$

Proof: Assume that the probability of each flow being assigned to any bucket is equal, and the allocation is independent. The number of flows in each bucket d_i follows a Poisson distribution with parameter $\frac{M}{D}$. According to the Chernoff bound, we have:

$$P(d_i > d) = P(d_i \geq (1 + \delta) * \frac{M}{D}) \leq \left(\frac{e^\delta}{(1 + \delta)^{(1 + \delta)}} \right)^{\frac{M}{D}} \quad (16)$$

The probability that at least one bucket has more than d flows:

$$P\left(\bigcup_{i=1}^D (d_i > d)\right) \leq D * P(d_i > d) \leq D \left(\frac{e^\delta}{(1 + \delta)^{(1 + \delta)}} \right)^{\frac{M}{D}} \quad (17)$$

D. False Positive Rate

Theorem 4: If a flow is a non-PI flow, then the upper bound of its probability of being identified as a PI is

$$\max\{\exp(-2(\omega - V * \rho)^2 / \nu), \exp(-\frac{1}{2}(1 - \frac{\omega}{\lambda})^2 * \lambda)\} \quad (18)$$

Proof: The probability that a flow is classified as PI:

$$\begin{aligned} P &= P\{(\nu \leq \omega) \&\& (N \geq \psi) \&\& (SMA \leq \phi)\} \leq \\ &P\{(\nu \leq \omega) \&\& (N \geq \psi)\} \leq \min\{P(\nu \leq \omega), P(N \geq \psi)\} \end{aligned} \quad (19)$$

For non-PI flows, one of two situations must exist: 1) The expectation of the ν $E\nu > \omega$. 2) The expectation of the number of windows N that appear $EN < \psi$.

• $P(\nu \leq \omega)$ Upper bound estimate

Let $\omega = (1 - \delta) * E\nu = (1 - \delta) * \lambda$, so:

$$\delta = 1 - \frac{\omega}{\lambda} \quad (20)$$

If $E\nu = \lambda > \omega$, then $\delta \in (0, 1)$. From the Chernoff bound we can derive:

$$P(\nu \leq \omega) = P(\nu \leq (1 - \delta) * E\nu) \leq \exp(-\frac{1}{2}(1 - \frac{\omega}{\lambda})^2 * \lambda) \quad (21)$$

• $P(N \geq \psi)$ Upper bound estimate

Considering Hoeffding Inequality, let $\tau = \psi - EN = \psi - V * \rho$, then

$$\begin{aligned} P(N \geq \psi) &= P(N - V * \rho) = P(N - V * \rho) \geq \tau \\ &\leq \exp(-2 * (\psi - V * \rho)^2 / V) \end{aligned} \quad (22)$$

Non-PI flow must satisfy one of the upper bounds $P(\nu \leq \omega)$ and $P(N \geq \psi)$, thus the conclusion holds.

E. Error Bound of PI Flow Replacement

Theorem 5: After hashing each flow into a bucket, each bucket contains at most d flows. The probability that a PI flow is replaced at position d is:

$$P \leq \exp(-\frac{1}{2}(\frac{\lambda TD}{\alpha * N} - 1)^2 * \frac{N}{\lambda TD}) \quad (23)$$

Proof: Every time a packet of a flow appears, the ν of the d -th flow is updated by $\nu = \nu - \alpha$. Therefore, if the PI flow appears once and the number of other flows is less than $\frac{1}{\alpha}$, then other flows cannot replace the d -th PI flow. Because the frequency of PI flow in any window is λ , the PI flow appears once in an average of $\frac{1}{\lambda}$ time window.

Let Y_i represent each packet, where $P(Y_i = 1) = P_i$ means that the packet is hashed into the current bucket, assuming $P_i = \frac{1}{D}$. During this time, an average of $k = \frac{N}{\lambda V}$ flows appear, let $Y = Y_1 + Y_2 + \dots + Y_k$, Y_i is Poisson trials.

$$EY = \sum_{i=1}^k EY_i = \frac{N}{\lambda VD} \triangleq \mu \quad (24)$$

So,
$$P(f_o \geq \frac{1}{\alpha}) = P(Y \geq \frac{1}{\alpha}) = P(Y \geq (1 + \delta) * \mu) \quad (25)$$

where, f_o represents the frequency of other streams, and $\delta = \frac{\lambda VD}{\alpha N} - 1$. From the Chernoff bound of the multiplicative form:

$$P \leq \exp(-\frac{1}{2}\delta^2\mu) = \exp(-\frac{1}{2}(\frac{\lambda VD}{\alpha * N} - 1)^2 * \frac{N}{\lambda VD}) \quad (26)$$

V. EXPERIMENTAL RESULTS

A. Experimental Setup

Implementation: The algorithm is implemented in C++, with experiments conducted on a server featuring dual Intel(R) Xeon(R) Gold 6230 CPU@2.10GHz processors and 256GB DRAM.

Datasets: Three real-world datasets are used, each containing approximately 5 million packets. The number of total flows and PI flows are shown in Table III.

TABLE III: The datasets used in the paper

Dataset	Total Packets	Total Flows	PI Plows
CAIDA 2018 [16]	5M	276867	1923
MAWI [17]	5M	393915	1383
Network [18]	5M	192397	2356

WisePIFinder's parameter configurations are divided into two categories: parameters related to the definition of the problem and parameters related to the algorithm setting. For the parameters pertaining to problem definition, Based on the analysis of the PI flow characteristic, we set $\omega=20$, $\psi=100$, $\phi=\omega$. These parameters directly reflect the PI flow characteristics, avoiding experience-based value dependencies seen in other measurement tasks. Traffic is divided into 1000 time windows ($V=1000$), with the LocalCounter's M reset every 10 windows ($n=10$). For the parameters related to the algorithm, compared to PISketch (bucket depth 5, weight threshold 3000) and OnOff+CM Sketch solution, the memory space ranges from 100KB to 300KB. WisePIFinder adopts a bucket design

with depth 4, meaning that each bucket contains 4 entries. We set $n=10$, which means that the number of consecutive observation windows is 10.

B. Metrics

Our evaluation uses the following three key metrics:

- **F1 Score:** $2 * PR * RR / (PR + RR)$, where precision rate (PR) represents the proportion of correctly identified PI flows among reported flows, and recall rate (RR) represents the proportion of correctly identified PI flows among total PI flows.
- **Average Relative Error (ARE):** Let $\hat{N}_1, \hat{N}_2, \dots, \hat{N}_k$ be estimated window counts of reported flows, and $N_1, N_2, \dots, N_k$ be actual window counts, then:

$$ARE = \frac{1}{k} \sum_{j=1}^k \frac{|N_j - \hat{N}_j|}{N_j} \quad (27)$$

- **Throughput:** Algorithm execution efficiency measured in million operations per second (Mops).

C. Experimental Results

1) **Performance Evaluation:** We comprehensively evaluate WisePIFinder in four dimensions: Throughput, Recall, F1 Score, and ARE. The results are as follow:

- **Throughput:** WisePIFinder outperforms existing solutions in processing efficiency, as shown in Fig.5. On the CAIDA 2018 dataset, it achieves $1.66\times$ and $1.94\times$ the throughput of PISketch and OnOff+CM Sketch, respectively. On the MAWI dataset, it achieves $1.65\times$ and $1.76\times$, and on the Network dataset, $2.18\times$ and $2.01\times$.
- **Recall:** WisePIFinder significantly outperforms comparison solutions in the three datasets, as shown in Fig.6. Taking the Network dataset with 100KB memory configuration as an example, RR reach $1.75\times$ that of PISketch and $2.64\times$ that of OnOff+CM Sketch.
- **F1 Score:** Evaluation results show WisePIFinder achieves optimal F1 scores across all three datasets, with scores increasing as memory increases, as shown in Fig.7. Under 100KB configuration, it shows approximately 20% improvement over PISketch and 120% over OnOff+CM Sketch.
- **ARE:** WisePIFinder maintains ARE below 0.1 across all three datasets, showing superior performance that continues to improve with increased memory, as shown in Fig.8. Taking the Network dataset with 300KB configuration as an example, accuracy reaches $10\times$ that of PISketch and $250\times$ that of OnOff+CM Sketch.

2) **Impact of Bucket Depth:** As mentioned in § III, bucket depth d is a key parameter for WisePIFinder. As shown in Fig.9, we evaluate the impact of different d values (memory range 30KB to 300KB) on algorithm performance using the CAIDA 2018 dataset:

- **F1 Score:** Experiments show that F1 Score gradually increases with memory size. When $d \geq 3$, F1 scores tend to converge as memory increases. However, in low memory scenarios ($\leq 50KB$), results when $d = 3$ or 4 show clear advantages, while result when $d = 2$ performs the worst.

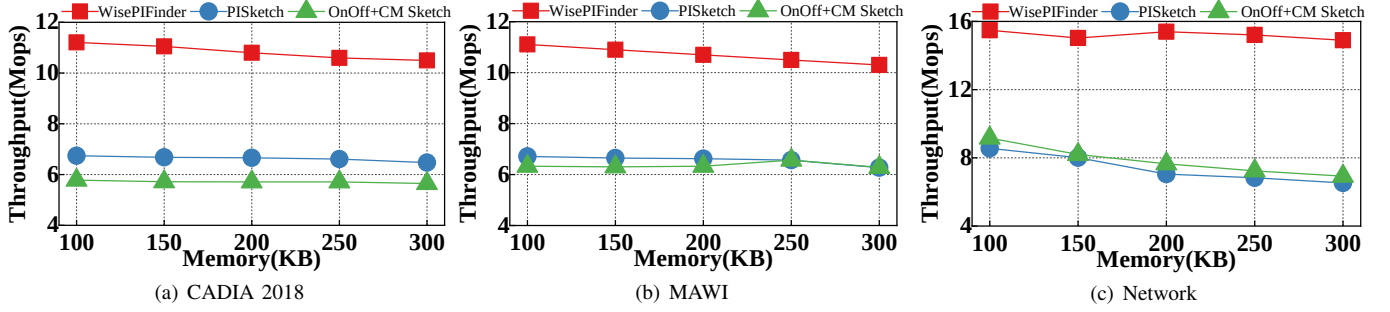


Fig. 5: Comparison of throughput on different real-world datasets.

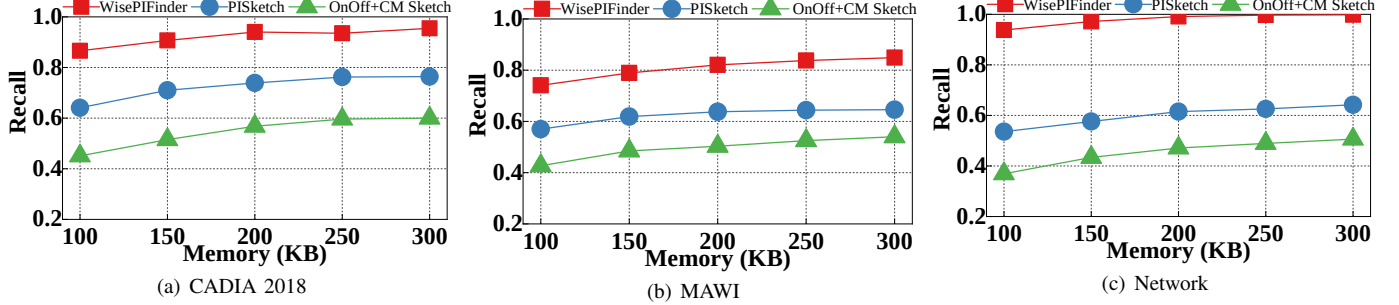


Fig. 6: Comparison of Recall on different real-world datasets.

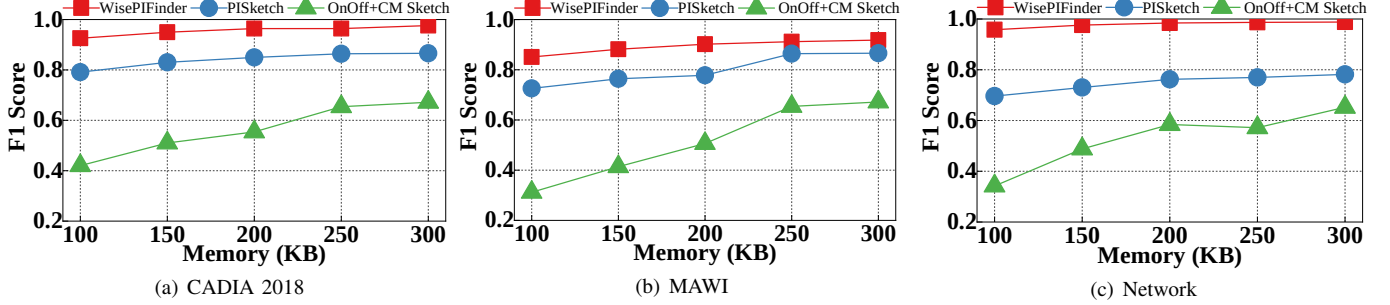


Fig. 7: Comparison of F1 Score on different real-world datasets.

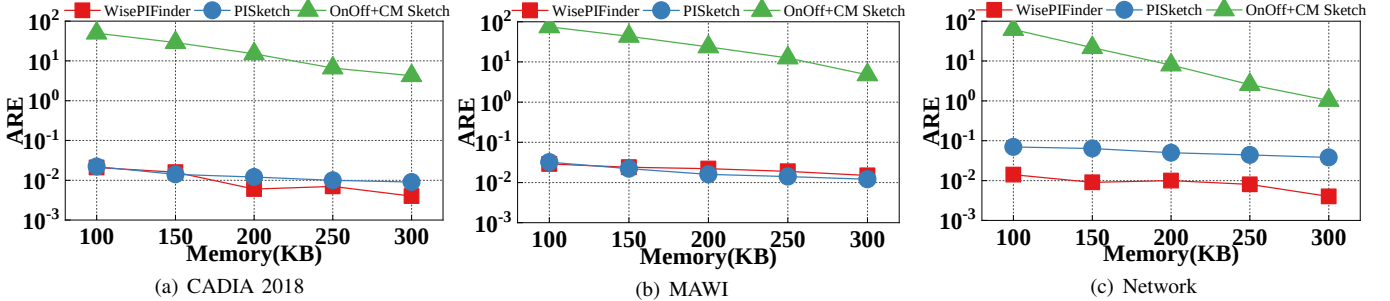


Fig. 8: Comparison of ARE on different real-world datasets.

- ARE: The differences between different d values are relatively small, but as increasing memory, the $d = 4$ configuration consistently maintains optimal performance.
- Throughput: Throughput exhibits a clear negative correlation with d value. Throughput at $d = 2$ is approximately $1.5\times$ that at $d = 5$.

3) **Impact of sliding window:** As mentioned in § III, sliding window n is a key parameter for WisePIFinder. As shown in Fig.10, we evaluate the impact of different n values (memory range 100KB to 300KB) on algorithm performance using the CAIDA 2018 dataset. The results for different values of n are similar. This is because the interval between two

occurrences of PI flow is usually relatively long.

D. Discussion

Despite PISketch's high accuracy, it suffers from significant issues with missed detections. Taking the CAIDA 2018 dataset as an example, among 270,000 flows (approximately 2,000 PI flows), PISketch missed over 500 PI flows. To better understand this phenomenon, we conduct a three-dimensional feature analysis of missed flows, as illustrated in Fig.11. Red balls represent PI flows missed by PISketch, with their projections onto the x, y, and z axes shown as purple, blue, and green dots, respectively. The x-axis represents the frequency, and the

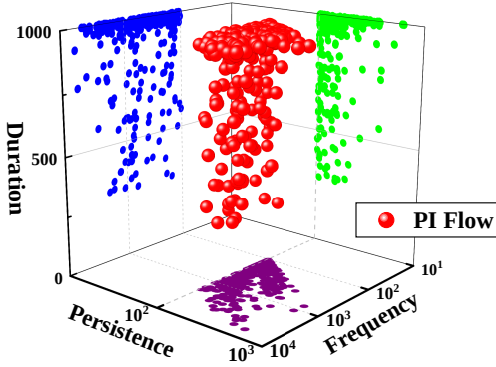
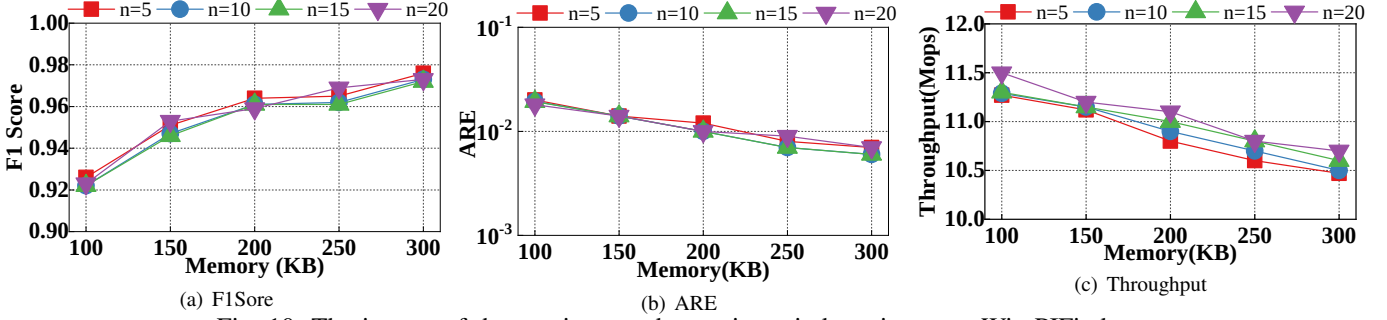
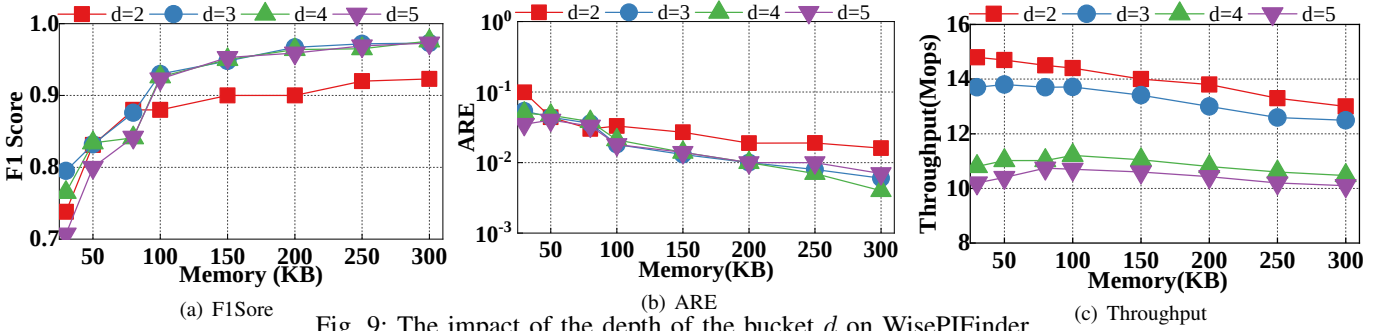


Fig. 11: The Loss of PISketch: Red balls indicate PI flows missed by PISketch. Purple dots show their frequency, green dots represent the total appearance windows, and blue dots denote flow duration (measured in consecutive windows).

y-axis shows the total number of appearance windows. The z-axis represents the duration, which is the difference between the time of the last occurrence and the first occurrence.

The blue projection (Frequency-Duration plane) reveals that many flows, despite having low total frequency, exhibit significant duration. Flows that last over 80% of total measurement time account for 86% of total missed detections—these are typical PI flows. The green projection (Persistence-Duration plane) shows that most missed flows, though appearing in fewer windows, have long durations. Specifically, 94% of missed flows have window counts between 100 and 272, indicating PISketch’s detection effectiveness highly depends on threshold selection.

In contrast, while WisePIFinder also sets a minimum window count threshold, it doesn’t affect measurement accuracy.

This is because, over time, non-persistent flows gradually distribute to hash bucket bottoms through continuous decay mechanisms, while PI flows migrate to the top of the heap. As long as there is sufficient memory space, PI flows will not be missed. The purple projection (Persistence-Frequency plane) corresponds exactly to the S_1 region described in Fig.2(a). WisePIFinder effectively solves PISketch’s missing detection problem through its strategy of “first ensuring persistence detection, then excluding flows not meeting APR or SMA conditions,” combined with heap storage strategy (persistent flows migrating to heap top) and decay mechanisms.

VI. RELATED WORK

Persistent and Infrequent Flow Lookup: PI flow detection, as an emerging network measurement task, is still in its early stages. PISketch [3] first proposed using a weighting mechanism to simultaneously characterize flow persistence and infrequency characteristics, providing a new approach to PI flow detection. However, while this simplified processing reduces task complexity, it inevitably affects accuracy. The field of PI flow detection still faces multiple challenges in algorithm design and performance optimization.

Persistent Flow Lookup: Multiple algorithms have been proposed for persistent flow detection. Traditional methods, such as Small-Space (SS) [19], rely on sampling to track item persistence with minimal space, while sketch-based methods are more efficient and accurate. On-Off Sketch [11] periodically updates item persistence but often misidentifies non-persistent items. It requires multiple hash functions, complex calculations, and consumes extra computational resources due to its SIMD acceleration. Pyramid-based On-Off Sketch [20] improves memory efficiency, but retains the original method’s flaws. WavingSketch [21] uses a Bloom Filter to remove

duplicates, but this leads to false positives and inaccuracies, especially in memory-limited scenarios. PIE [10] employs coding for tracking but suffers from a complex update process, hindering its use in constrained environments. P-Sketch [22] leverage arrival continuity and bucket stability to better identify persistent items. Additionally, based on persistent flow tasks, a series of other tasks [13] [23] have emerged, but they are difficult to apply directly to PI flow detection.

Frequency Estimation: Estimation of flow frequency is a fundamental problem in network measurement and data analysis. Sketch techniques have been widely proven to achieve high-precision estimation under limited memory constraints [9], [24]–[26]. Building on this foundation, researchers have proposed a series of improved algorithms [27]–[31], which primarily focus on improving estimation accuracy for large flows. In recent years, several network measurement systems designed for practical deployment have also emerged [24], [25], [32], [33].

VII. CONCLUSION

As a key pattern in network traffic, persistent and infrequent flows play a crucial role in network behavior analysis and anomaly detection. Through a comprehensive examination of the overall and local characteristics of PI flows, this study proposes a more complete feature description system and, based on this, designs the WisePIFinder algorithm. While maintaining low memory overhead, the algorithm successfully achieves dual objectives: high-precision detection and fast processing. We have validated the effectiveness of WisePIFinder through evaluations of multiple real-world network datasets. The results demonstrate significant improvements in both accuracy and throughput. With moderate adjustments to its data structure, WisePIFinder can also be adapted for other persistence-related tasks.

REFERENCES

- [1] R. Xu, W. Li, K. Li, X. Zhou, and H. Qi, "Darkte: towards dark traffic engineering in data center networks with ensemble learning," in *ACM IWQoS*, 2021, pp. 1–10.
- [2] Z. Fan, Z. Hu, Y. Wu, J. Guo, W. Liu, T. Yang, H. Wang, Y. Xu, S. Uhlig, and Y. Tu, "Pisketch: finding persistent and infrequent flows," in *ACM SIGCOMM Workshop*, 2022, pp. 8–14.
- [3] Z. Fan, Z. Hu, Y. Wu, J. Guo, S. Wang, W. Liu, T. Yang, Y. Tu, and S. Uhlig, "Pisketch: Finding persistent and infrequent flows," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3191–3206, 2023.
- [4] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019.
- [5] J. Hou, X. Li, K. Zhao, W. Liang, Y. Li, T. Jiang, and Z. Liu, "A survey of dns tunnel detection," in *ICSIP*, 2022, pp. 338–342.
- [6] F. Zhang, C. Zhang, J. Guan, Q. Zhou, K. Chen, X. Zhang, B. He, J. Zhai, and X. Du, "Breaking the edge: Enabling efficient neural network inference on integrated edge devices," *IEEE Transactions on Cloud Computing*, pp. 1–17, 2025.
- [7] F. Zhang, J. Zhai, X. Shen, O. Mutlu, and X. Du, "Poclib: A high-performance framework for enabling near orthogonal processing on compression," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 2, pp. 459–475, 2022.
- [8] J. Neuberg, R. Luckett, B. Baptie, and K. Olsen, "Models of tremor and low-frequency earthquake swarms on montserrat," *Journal of Volcanology and Geothermal Research*, vol. 101, no. 1, pp. 83–104, 2000.
- [9] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.
- [10] H. Dai, M. Shahzad, A. X. Liu, M. Li, Y. Zhong, and G. Chen, "Identifying and estimating persistent items in data streams," *IEEE/ACM Trans. Netw.*, vol. 26, no. 6, pp. 2429–2442, 2018.
- [11] Y. Zhang, J. Li, Y. Lei, T. Yang, Z. Li, G. Zhang, and B. Cui, "On-off sketch: a fast and accurate sketch on persistence," *Proc. VLDB Endow.*, vol. 14, no. 2, p. 128–140, Oct. 2020.
- [12] (2025) Source code and more details related to wisepifinder. [Online]. Available: <https://anonymous.4open.science/r/WisePIFinder-F9CC/>
- [13] X. Li, Z. Fan, H. Li, Z. Zhong, J. Guo, S. Long, T. Yang, and B. Cui, "Steadysketch: Finding steady flows in data streams," in *IEEE/ACM IWQoS*, 2023, pp. 01–09.
- [14] M. Parkour. (2013) Contagio malware dataset. [Online]. Available: <https://www.mediafire.com/folder/>
- [15] D. M. W. Powers, "Applications and explanations of zipf's law," in *NeMLaP3/CoNLL*, 1998, p. 151–160.
- [16] (2018) The caida anonymized internet traces. [Online]. Available: <https://www.caida.org/catalog/datasets/overview/>
- [17] (2010) Mawi working group traffic archive. [Online]. Available: <http://mawi.wide.ad.jp/mawi/>
- [18] (2014) The network dataset internet traces. [Online]. Available: <http://snap.stanford.edu/data/>
- [19] S. Gandhi, L. Foschini, and S. Suri, "Space-efficient online approximation of time series data: Streams, amnesia, and out-of-order," in *IEEE ICDE*, 2010, pp. 924–935.
- [20] Y. Li, X. Yu, Y. Yang, Y. Zhou, T. Yang, Z. Ma, and S. Chen, "Pyramid family: Generic frameworks for accurate and fast flow size measurement," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 586–600, 2022.
- [21] J. Li, Z. Li, Y. Xu, S. Jiang, T. Yang, B. Cui, Y. Dai, and G. Zhang, "Wavingsketch: An unbiased and generic sketch for finding top-k items in data streams," in *ACM SIGKDD*, 2020, p. 1574–1584.
- [22] W. Li and P. Patras, "P-sketch: A fast and accurate sketch for persistent item lookup," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 987–1002, 2024.
- [23] T. Ma, G. Gao, H. Huang, Y.-E. Sun, and Y. Du, "Scout sketch: Finding promising items in data streams," in *IEEE INFOCOM*, 2024, pp. 1561–1570.
- [24] Z. Liu, R. Ben-Basat, G. Einziger, Y. Kassner, V. Braverman, R. Friedman, and V. Sekar, "Nitrosketch: robust and general sketch-based monitoring in software switches," in *SIGCOMM*, 2019, p. 334–350.
- [25] Y. Zhang, Z. Liu, R. Wang, T. Yang, J. Li, R. Miao, P. Liu, R. Zhang, and J. Jiang, "Cocosketch: high-performance sketch-based measurement over arbitrary partial key query," in *ACM SIGCOMM*, 2021, p. 207–222.
- [26] R. Schweller, Z. Li, Y. Chen, Y. Gao, A. Gupta, Y. Zhang, P. A. Dinda, M.-Y. Kao, and G. Memik, "Reversible sketches: enabling monitoring and analysis over high-speed data streams," *IEEE/ACM Trans. Netw.*, vol. 15, no. 5, p. 1059–1072, Oct. 2007.
- [27] X. Gou, Y. Zhang, Z. Hu, L. He, K. Wang, X. Liu, T. Yang, Y. Wang, and B. Cui, "A sketch framework for approximate data stream processing in sliding windows," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4411–4424, 2023.
- [28] Y. Li, X. Yu, Y. Yang, Y. Zhou, T. Yang, Z. Ma, and S. Chen, "Pyramid family: Generic frameworks for accurate and fast flow size measurement," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 586–600, 2022.
- [29] T. Yang, H. Zhang, J. Li, J. Gong, S. Uhlig, S. Chen, and X. Li, "Heavykeeper: An accurate algorithm for finding top- k elephant flows," *IEEE/ACM Trans. Netw.*, vol. 27, no. 5, pp. 1845–1858, 2019.
- [30] T. Yang, J. Gong, H. Zhang, L. Zou, L. Shi, and X. Li, "Heavyguardian: Separate and guard hot items in data streams," in *ACM SIGKDD*, 2018, p. 2584–2593.
- [31] Y. Zhou, T. Yang, J. Jiang, B. Cui, M. Yu, X. Li, and S. Uhlig, "Cold filter: A meta-framework for faster and more accurate stream processing," in *ACM SIGMOD*, 2018, p. 741–756.
- [32] Y. Zhao, K. Yang, Z. Liu, T. Yang, L. Chen, S. Liu, N. Zheng, R. Wang, H. Wu, Y. Wang, and N. Zhang, "LightGuardian: A Full-Visibility, lightweight, in-band telemetry system using sketchlets," in *NSDI*, Apr. 2021, pp. 991–1010.
- [33] H. Zheng, C. Huang, X. Han, J. Zheng, X. Wang, C. Tian, W. Dou, and G. Chen, "μmon: Empowering microsecond-level network monitoring with wavelets," in *ACM SIGCOMM*, 2024, p. 274–290.